

Ocena możliwości modeli LLM w automatycznym wspieraniu aktywności cyklu życia oprogramowania

Projekt koncentruje się na opracowaniu i eksperymentalnym zbadaniu możliwości dużych modeli językowych (LLM) w automatycznym wspieraniu poszczególnych aktywności cyklu życia oprogramowania (Software Development Life Cycle). Celem jest zrozumienie, w jakim zakresie nowoczesne modele – takie jak GPT-4.1, Claude 3.5 Sonnet czy Llama 3.1 – mogą realnie wspomagać inżynierów w zadaniach analitycznych, projektowych, implementacyjnych i testowych.

Ideą projektu jest stworzenie wieloaspektowego benchmarku obejmującego zadania reprezentujące kluczowe obszary SDLC oraz porównanie jakości odpowiedzi różnych modeli. Pozwoli to ocenić ich przydatność, ograniczenia oraz potencjalne ryzyka związane z wykorzystaniem w procesach inżynierii oprogramowania.

Możliwe kierunki realizacji obejmują:

- * Analiza wymagań – wykrywanie niejednoznaczności, doprecyzowanie funkcjonalności, generowanie user stories i kryteriów akceptacji.
- * Modelowanie i architektura – tworzenie opisów architektury, dekompozycja modułów, analiza API i rekomendacje wzorców projektowych.
- * Generowanie i refaktoring kodu – implementacja funkcji na podstawie opisu, poprawa jakości kodu, wykrywanie błędów oraz uzasadnianie zmian.
- * Projektowanie testów – tworzenie przypadków testowych (w tym brzegowych), generowanie testów jednostkowych i analiza zgłoszonych błędów.
- * Wsparcie DevOps / CI-CD – analiza konfiguracji pipeline, wykrywanie problemów w YAML, dobre praktyki bezpieczeństwa i interpretacja logów.

Opracowanie metodyki oceny jakości – opracowanie jednolitego systemu

scoringu (0–3), zastosowanie testów automatycznych oraz ewaluacja odpowiedzi za pomocą LLM-sędziego.

Projekt pozwala zrozumieć, które obszary cyklu życia są najbardziej

podatne na automatyzację z wykorzystaniem Gen AI, a gdzie modele językowe

wykazują istotne ograniczenia. Może obejmować implementację benchmarku SDLC-LLM, porównanie modeli, analizę błędów oraz propozycje rekomendacji i dobrych praktyk wykorzystania LLM w projektach informatycznych.

Główne pytanie badawcze:

RQ0:

How do LLM-assisted workflows affect the quality, consistency, and error propagation across the Software Development Life Cycle compared to human-only and hybrid approaches?

Pytania szczegółowe uzależnione od tasku:

Requirements Team

RQ1:

How accurately do LLMs interpret and refine ambiguous requirements compared to humans?

RQ2:

What types of ambiguities are most frequently misinterpreted by LLMs?

Architecture Team

RQ3:

How do LLM-generated architectures compare to human-designed ones in terms of correctness, modularity, and scalability?

RQ4:

To what extent do architectural decisions made by LLMs align with initial requirements?

Coding & Refactoring Team

RQ5:

What is the functional correctness and maintainability of LLM-generated code compared to human-written code?

RQ6:

How effectively do LLMs perform code refactoring and bug fixing?

Testing Team

RQ7:

How effective are LLM-generated test cases in detecting defects compared to human-designed tests?

RQ8:

Do LLM-generated tests adequately cover the original requirements?

DevOps Team

RQ9:

How reliable are LLM-generated CI/CD configurations and deployment scripts?

RQ10:

Can LLMs correctly diagnose and resolve pipeline failures?

Evaluation Team (klucz, najważniejsze)

RQ11 (najważniejsze):

How do errors introduced in earlier SDLC phases propagate to later stages in LLM-assisted workflows?

RQ12:

How does cross-phase consistency (requirements ↔ design ↔ code ↔ tests) differ between human, LLM, and hybrid approaches?

RQ13:

Which SDLC phases benefit most from LLM support, and which remain bottlenecks?

Hipotezy:

H1 (najważniejsza)

Errors introduced by LLMs in early SDLC phases propagate and amplify in later stages more than in human-driven workflows.

H2

LLM-assisted workflows achieve higher speed but lower cross-phase consistency than human-only workflows.

H3

Hybrid (human + LLM) workflows outperform both human-only and LLM-only approaches in overall quality and reliability.

H4

LLMs perform better in implementation and testing tasks than in requirements analysis and architectural design.

H5

LLM-generated tests have high surface coverage but lower fault-detection effectiveness compared to human-designed tests.

H6

LLM-generated architectures tend to be syntactically correct but suboptimal in modularity and long-term maintainability.

H7 (bardzo publikacyjne)

LLMs struggle with maintaining semantic consistency across SDLC artifacts (requirements, code, tests).

H8

Iterative (feedback-loop) use of LLMs improves local quality but does not fully eliminate systemic errors.

Zadania i eksperymenty dla poszczególnych tasków w SDLC

Każdy zespół robi 2 eksperymenty:

E1: LOCAL (izolowany)

- dane: przygotowane („gold standard”)
- cel: ocena realnych możliwości LLM

E2: PIPELINE

- dane: z poprzedniego zespołu
- cel: analiza propagacji błędów

1. REQUIREMENTS TEAM (Paweł Adamczyk, Konrad Cwięka, I stopień)

E1 – LOCAL

Cel Ocena, czy LLM poprawnie analizuje i doprecyzowuje wymagania

Dane (Swoje)

- przygotowane opisy systemów:
 - częściowo niejednoznaczne
 - częściowo niekompletne

Zadania

- wykrycie:
 - niejednoznaczności
 - braków
 - konfliktów
- wygenerowanie:
 - user stories
 - acceptance criteria

Metryki

- liczba wykrytych problemów (vs gold)
- precision/recall wykrytych niejednoznaczności
- kompletność wymagań
- poprawność semantyczna

Output

- zestaw „oczyszczonych” wymagań
- lista problemów

E2 – PIPELINE

Cel Zbadanie, jak jakość wymagań wpływa na dalsze etapy

Dane

- brak (to pierwszy etap pipeline)

Zadania

- wygenerować wymagania w 3 trybach:
 - human
 - LLM

- hybrid

Output

- 3 wersje wymagań → idą do Architecture team

2. ARCHITECTURE TEAM (Kacper Drożdż, Eryk Hadała, II stopień)

E1 – LOCAL

Cel Ocena jakości architektur generowanych przez LLM

Dane (Swoje)

- gold requirements (poprawne, referencyjne)

Zadania

- zaprojektować:
 - architekturę
 - podział na moduły
 - API
- mapować wymagania → komponenty

Metryki

- zgodność z wymaganiami
- modularność
- kompletność
- jakość decyzji architektonicznych

Output

- architektura referencyjna + LLM/human/hybrid

E2 – PIPELINE

Cel Sprawdzenie wpływu jakości wymagań na architekturę

Dane (z Requirements team)

- wymagania (3 warianty: human / LLM / hybrid)

Zadania

- zaprojektować architekturę dla każdej wersji

Metryki

- degradacja jakości względem E1
- liczba błędnych decyzji wynikających z wymagań

Output

- architektury → idą do Coding team

3. CODING & REFACTORING TEAM (Mateusz Bielówka, Maksim Dziatkou, I stopień)

E1 – LOCAL

Cel Ocena jakości kodu generowanego przez LLM

Dane (Swoje)

- gold requirements + gold architecture

Zadania

- implementacja funkcjonalności
- refaktoryzacja kodu
- naprawa bugów

Metryki

- poprawność funkcjonalna
- liczba błędów
- maintainability (np. complexity)
- jakość refaktoryzacji

Output

- kod referencyjny + warianty

E2 – PIPELINE

Cel Sprawdzenie wpływu architektury na jakość kodu

Dane (z Architecture team)

- architektury (różnej jakości)

Zadania

- implementacja na ich podstawie

Metryki

- liczba błędów wynikających z designu
- zgodność z wymaganiami
- degradacja jakości vs E1

Output

- kod → idzie do Testing team

4. TESTING TEAM (Joanna Jochimczyk, Zuzanna Konopka, II stopień)

E1 – LOCAL

Cel Ocena jakości testów generowanych przez LLM

Dane (Swoje)

- gold code + gold requirements

Zadania

- generowanie:
 - testów jednostkowych
 - przypadków testowych
- mapowanie testów do wymagań

Metryki

- coverage (code + requirements)
- defect detection rate
- liczba edge cases

Output

- zestaw testów referencyjnych

E2 – PIPELINE

Cel Sprawdzenie skuteczności testów na „realnym” kodzie

Dane (z Coding team)

- kod (różnej jakości)

Zadania

- generowanie testów
- uruchamianie testów
- wykrywanie błędów

Metryki

- wykryte vs niewykryte błędy
- false negatives
- pokrycie wymagań

Output

- raport testów → do Evaluation team

5. DEVOPS TEAM (Karol Bysterk, Patryk Chamera, I stopień)

E1 – LOCAL

Cel Ocena jakości pipeline'ów generowanych przez LLM

Dane (Swoje)

- gold projekt (kod + testy)

Zadania

- stworzenie CI/CD
- analiza bezpieczeństwa
- interpretacja logów

Metryki

- poprawność pipeline
- liczba błędów konfiguracyjnych
- security issues

Output

- pipeline + raport jakości

E2 – PIPELINE

Cel

Sprawdzenie działania pipeline na „realnych” artefaktach

Dane (z Testing team)

- kod + testy (różnej jakości)

Zadania

- uruchomienie pipeline
- diagnoza błędów
- poprawki

Metryki

- liczba błędów pipeline
- skuteczność diagnozy
- czas naprawy

Output

- logi + diagnozy → do Evaluation team

Ogólna uwaga: można się ograniczyć do GitHub

6. EVALUATION TEAM

E1 – LOCAL ANALYSIS

Cel Porównanie jakości per etap

Dane

- wszystkie wyniki z E1 (każdy zespół)

Zadania

- stworzenie wspólnego scoringu
- porównanie:
 - human vs LLM vs hybrid

Metryki

- correctness
- completeness
- maintainability
- security

E2 – PIPELINE ANALYSIS

Cel Analiza systemowa (core paper)

Dane

- pełny pipeline:
 - requirements → arch → code → test → devops

Zadania

- analiza:
 - propagacji błędów
 - spójności międzyfazowej
- śledzenie:
 - requirement → code → test

Metryki

- consistency score
- error propagation rate
- final system quality

Różne case study do przejścia przez pipeline:

Case Study 1: System rezerwacji wizyt (Appointment Booking System)

- klasyczny system biznesowy
- dużo logiki + edge cases
- łatwo wprowadzić niejednoznaczności

- dobry do testowania consistency (requirements ↔ code ↔ tests)

Opis (dla Requirements team – wersja surowa)

System umożliwi użytkownikom rezerwację wizyt u specjalistów (np. lekarzy lub konsultantów).

Użytkownicy mogą przeglądać dostępne terminy, dokonywać rezerwacji oraz je anulować.

Specjaliści zarządzają swoim grafikiem.

System powinien zapobiegać konfliktom rezerwacji i obsługiwać różne role użytkowników.

celowo: brak szczegółów (np. anulowanie, overbooking, strefy czasowe)

Co testuje w SDLC

- Requirements: niejednoznaczności (np. co z anulacją w ostatniej chwili?)
- Architecture: concurrency, state management
- Coding: walidacje, logika biznesowa
- Testing: edge cases (double booking)
- DevOps: API + baza + testy integracyjne

Potencjalne trudności

- równoczesne rezerwacje
- konflikty czasowe
- role (user/admin)
- anulacje i refundy

Pipeline artefakty

- wymagania → architektura (np. REST API) → backend → testy → pipeline CI

Case Study 2: System zgłoszeń (Helpdesk / Ticketing System)

- dużo pracy na tekstach (idealne dla LLM)
- workflow + stany + role
- świetne do testowania logiki i consistency

Opis (surowy)

System do zarządzania zgłoszeniami użytkowników.

Użytkownicy mogą tworzyć zgłoszenia, które są obsługiwane przez agentów.

Zgłoszenia mają statusy i mogą być przypisywane do różnych osób.

System powinien umożliwiać śledzenie historii oraz komunikację między użytkownikiem a agentem.

niejasności:

- jakie statusy?
- czy można reopen ticket?
- SLA?
- wiele agentów?

Co testuje

- Requirements: interpretacja workflow
- Architecture: state machine
- Coding: logika statusów
- Testing: przejścia stanów
- DevOps: logi, monitoring

Trudności

- zarządzanie stanami (open, in progress, closed, reopened)
- przypisywanie agentów
- historia zmian
- edge cases (np. zamknięty ticket z nową odpowiedzią)

Pipeline artefakty

- workflow → model stanów → implementacja → testy stanów → CI/CD

Case Study 3: API do zarządzania plikami (File Storage Service)

- bardziej techniczne
- świetne dla DevOps i architektury
- testuje bezpieczeństwo i edge cases

Opis (surowy)

System udostępnia API do przesyłania, pobierania i usuwania plików.
Użytkownicy powinni mieć dostęp tylko do swoich plików.
System powinien być skalowalny i bezpieczny.

niejasności:

- limity plików
- autoryzacja
- wersjonowanie
- duże pliki

Co testuje

- Requirements: security requirements
- Architecture: storage, API, auth
- Coding: obsługa plików, walidacje
- Testing: edge cases (duże pliki, błędy uploadu)
- DevOps: deployment, konfiguracje, bezpieczeństwo

Trudności

- autoryzacja
- zarządzanie plikami
- błędy sieciowe
- bezpieczeństwo (np. dostęp do czyjych plików)

Pipeline artefakty

- API spec → architektura → implementacja → testy → deployment

Uwaga:

- Pipeline Te same 3 case studies przechodzą przez wszystkie zespoły
- Local experiments Każdy zespół może mieć dodatkowe mini-taski

Oceny (np. scoring 0–3)

0 – nieakceptowalne

- wynik jest błędny, niekompletny albo bezużyteczny

- nie spełnia podstawowego celu zadania
- wymaga zasadniczego przepisania

1 – słabe

- częściowo poprawne, ale z istotnymi brakami lub błędami
- może zawierać sensowne elementy, ale nie nadaje się do użycia bez dużych poprawek

2 – dobre

- w większości poprawne i użyteczne
- są drobne błędy, braki albo niedociągnięcia
- wymaga umiarkowanych poprawek

3 – bardzo dobre

- poprawne, kompletne, spójne i praktycznie gotowe do użycia
- ewentualne uwagi mają charakter kosmetyczny

Ważne: ta sama logika 0–3 musi działać wszędzie.

Metryki wspólne dla wszystkich zespołów

Kryteria, które można stosować do każdego artefaktu: wymagań, architektury, kodu, testów i DevOps.

M1. Correctness (poprawność)

Czy artefakt jest merytorycznie poprawny?

- 0: w większości błędny
- 1: częściowo poprawny, ale zawiera istotne błędy
- 2: głównie poprawny, drobne błędy
- 3: poprawny

M2. Completeness (kompletność)

Czy obejmuje wszystkie istotne elementy zadania?

- 0: brakuje większości kluczowych elementów
- 1: brakuje kilku istotnych elementów
- 2: prawie kompletny

- 3: kompletny

M3. Consistency (spójność)

Czy artefakt jest wewnętrznie spójny i zgodny z wejściem z poprzedniego etapu?

- 0: liczne sprzeczności lub rozjazdy
- 1: zauważalne niespójności
- 2: zasadniczo spójny, drobne problemy
- 3: spójny

M4. Clarity / Explainability (jasność, czytelność)

Czy artefakt jest czytelny i zrozumiały dla inżyniera?

- 0: nieczytelny lub chaotyczny
- 1: trudny w interpretacji
- 2: raczej czytelny
- 3: bardzo czytelny i dobrze uzasadniony

M5. Maintainability / Practical usefulness (utrzymywalność / użyteczność praktyczna)

Czy wynik nadaje się do realnego użycia w projekcie?

- 0: praktycznie nieużywalny
- 1: wymaga dużej przebudowy
- 2: używalny po poprawkach
- 3: gotowy lub prawie gotowy do użycia

Wytyczne dla zadań lokalnych (non-pipeline)

1. Jedna kompetencja na zadanie
→ każde zadanie testuje jeden aspekt (np. tylko testy, tylko architektura)
2. Wspólne, kontrolowane dane wejściowe
→ wszystkie grupy dostają dokładnie to samo
3. Jasny, mierzalny output
→ artefakt, który da się ocenić (kod, testy, wymagania, raport)
4. Z góry zdefiniowane metryki (0–3)
→ brak oceny „na oko”

5. Umiarkowana złożoność
→ nie trywialne, ale też nie mini-projekt
6. Realizm inżynierski
→ zadania jak w praktyce (bug fix, testy, CI, wymagania)
7. Powtarzalność i porównywalność
→ umożliwia porównanie: human vs LLM vs hybrid

REQUIREMENTS TEAM

Zadanie R1 – Ambiguity detection

Cel: wykrywanie niejednoznaczności

Wejście: System umożliwia rezerwację wizyt. Użytkownicy mogą anulować wizyty i zarządzać nimi.

Zadanie: wypisz wszystkie niejednoznaczności, braki, pytania

Output: lista problemów + pytania do doprecyzowania

Mierzyć: R1 (ambiguity detection), Correctness, Completeness

Zadanie R2 – Refinement

Wejście: krótki opis systemu (jak wyżej)

Zadanie:

- utwórz:
 - user stories
 - acceptance criteria

Output: uporządkowane wymagania

Mierzyć:

- R2 (quality), R3 (acceptance criteria), Clarity

Zadanie R3 – Conflict detection

Wejście:

User może anulować wizytę w dowolnym momencie.
Anulowanie możliwe tylko 24h wcześniej.

Zadanie: wykryj konflikty i zaproponuj poprawkę

Output: lista konfliktów + poprawiona wersja

Mierzyć: Correctness, Consistency, R1

ARCHITECTURE TEAM

Zadanie A1 – Architecture design

Wejście: gold requirements (booking system)

Zadanie:

- zaprojektuj:
 - komponenty
 - API
 - przepływ danych

Output: opis architektury + diagram (tekstowy ok)

Mierzyć:

- A1 (alignment), A2 (modularity), A3

♦ Zadanie A2 – Pattern selection

Wejście: system z wieloma użytkownikami i rezerwacjami

Zadanie:

- wybierz wzorce (np. MVC, microservices)
- uzasadnij

Output: lista decyzji + uzasadnienie

Mierzyć:

- A3 (soundness), Clarity

Zadanie A3 – Architecture critique

Wejście: celowo słaba architektura (np. monolit bez warstw)

Zadanie:

- znajdź problemy
- zaproponuj poprawki

Output: lista problemów + ulepszenia

Mierzyć: Correctness, A3, Maintainability

CODING & REFACTORING TEAM

Zadanie C1 – Code generation

Wejście:

Implement function:

`book_slot(user_id, slot_id)`

Rules:

- slot must be available
- max 3 bookings per user

Zadanie: napisz kod

Output: funkcja

Mierzyć: C1 (correctness), C2 (quality)

Zadanie C2 – Bug fixing

Wejście: kod z bugiem (np. brak sprawdzenia limitu 3)

Zadanie: znajdź i napraw błąd

Output: poprawiony kod + opis

Mierzyć: C4 (bug fixing), Correctness

Zadanie C3 – Refactoring

Wejście: brzydki kod (duplikacje, brak funkcji)

Zadanie: popraw strukturę bez zmiany logiki

Output: refactored code

Mierzyć: C3, Maintainability

TESTING TEAM

Zadanie T1 – Test generation

Wejście: funkcja booking (poprawna)

Zadanie: napisz testy jednostkowe

Output: test suite

Mierzyć: T1 (coverage), T4 (quality)

Zadanie T2 – Edge cases

Wejście: opis systemu booking

Zadanie: zaproponuj edge cases

Output: lista przypadków testowych

Mierzyć: T3 (edge cases), Completeness

Zadanie T3 – Fault detection

Wejście: kod z ukrytym błędem

Zadanie: napisz testy, które go wykryją

Output: test + wynik

Mierzyć: T2 (fault detection)

DEVOPS TEAM

Zadanie D1 – Pipeline creation

Wejście: repo z kodem + testami

Zadanie: napisz CI pipeline (build + test)

Output: YAML

Mierzyć: D1 (correctness)

Zadanie D2 – Debugging

Wejście: błędny YAML

Zadanie: znajdź i napraw problem

Output: poprawiony YAML + diagnoza

Mierzyć: D2 (diagnostic quality)

Zadanie D3 – Log analysis

Wejście: log błędu z pipeline

Zadanie: zdiagnozuj problem

Output: przyczyna + rozwiązanie

Mierzyć: D2, Clarity

EVALUATION TEAM

Zadanie E1 – Consistency check

Wejście: wymagania, kod, testy

Zadanie:

- sprawdź:
 - czy kod realizuje wymagania
 - czy testy pokrywają wymagania

Output: raport spójności

Mierzyć:

- E1 (consistency)

Zadanie E2 – Comparative evaluation

Wejście:

3 wersje (human / LLM / hybrid)

Zadanie: porównaj jakość

Output: tabela wyników

Mierzyć: wszystkie metryki globalne

Zadanie E3 – Error analysis

Wejście: artefakty z błędami

Zadanie:

- sklasyfikuj błędy:
 - typ
 - źródło

Output: typologia błędów

Mierzyć: E2 (propagation insight)

Ustalenia operacyjne

1. Gold standard

Dla każdego case study przygotowane zostaną referencyjne: wymagania, architektura, kod i testy. Służą wyłącznie do oceny.

2. Protokół eksperymentu
Każdy zespół wykonuje E1 (local) i E2 (pipeline). Artefakty są przekazywane między zespołami w ustalonej kolejności SDLC.
3. Zasady użycia LLM
Maks. 3 iteracje na zadanie. Prompty i odpowiedzi muszą być zapisane.
4. Standaryzacja środowiska
Wszyscy używają tych samych modeli LLM i ustawień (np. temperatura = 0.2).
5. Format raportowania
Każde zadanie: wejście, output, metryki (0–3), krótka analiza. Wyniki w tabeli.
6. Definicja trybu hybrid
LLM generuje rozwiązanie, człowiek może je poprawić. Zakres zmian musi być opisany.
7. Zbieranie danych
Zbierane są: artefakty, metryki, logi promptów.
8. Ograniczenia
Wyniki zależą od modelu, promptów i case study.

Modele LLM:

GPT (OpenAI)

Claude (Anthropic)

Gemini (Google)

Llama (open-weight)

Pracujemy na wersja najlepszych, ale ogólnie dostępnych, niekomercyjnych.

W razie trudności ilościowych eksperymentów ograniczamy się do dwóch modeli:

GPT i Gemini, np.: **GPT-5.4**, **Gemini 2.5 Pro**

Należy przyjąć 3 tryby:

1. Human-only
2. LLM-only
3. Hybrid (human + LLM)

Wolałbym zobaczyć wszystkie trzy tryby, w ostateczności redukcja do 2 i 3.

